



Universitat
de les Illes Balears

TRABAJO DE FIN DE GRADO

SPORTS LEAGUE: DISEÑO E IMPLEMENTACIÓN DE UN SISTEMA PARA LA GESTIÓN DE LIGAS DEPORTIVAS

Carlos Gálvez Mena

Grau d'Enginyeria Informàtica

Escola Politècnica Superior

Año académico 2025-26

SPORTS LEAGUE: DISEÑO E IMPLEMENTACIÓN DE UN SISTEMA PARA LA GESTIÓN DE LIGAS DEPORTIVAS

Carlos Gálvez Mena

Trabajo de Fin de Grado

Escola Politècnica Superior

Universitat de les Illes Balears

Año académico 2025-26

Paraules clau del treball: Liga deportiva, Aplicación multiplataforma, Spring Boot, React Native, TFG

Tutor: Antonio Crespí Tobeña

Gracias a todos los profesores que me han ayudado a lo largo de mi carrera, y en especial a mi primer tutor Raimon Massanet Vila por haber confiado en mi idea y, mi segundo tutor, Antonio Crespí Tobeña por su apoyo y orientación durante la realización de este trabajo, aun habiendo tenido que coger el relevo cuando el primer tutor tuvo que ser relevado por motivos burocráticos.

ÍNDICE GENERAL

Índice general	iii
Acrònims	v
Resumen	vii
1 Introducción	1
1.1 Estructura del documento	1
1.2 Contexto	1
1.3 Motivación y objetivos	2
1.4 Alternativas actuales	2
2 Estado del arte en el ámbito tecnológico	5
2.1 Tecnologías utilizadas	6
3 Proceso de desarrollo	7
3.1 Análisis y diseño	7
3.1.1 Recogida de requisitos	7
3.2 Análisis	8
3.2.1 Reunión con la OUSIS	8
3.2.2 Requisitos funcionales	8
3.3 Diseño	18
3.3.1 Diseño de la BD	18
3.3.2 Diseño de la interfaz	19
3.3.3 Diseño del contrato API	19
3.4 Arquitectura del sistema	19
3.5 Planificación	19
3.6 Implementación	19
4 Conclusiones	21
4.1 Aprendizaje	21
4.2 Aporte	21
4.3 Trabajo futuro	21
A Apéndice	23
A.1 Formularios de opinión	23
A.2 Diseño base de datos	23
A.3 Contrato API	23

Bibliografía

25

ACRÒNIMS

TFG Treball Final de Grau

GEIN Grau d'Enginyeria Informàtica

UIB Universitat de les Illes Balears

OUSIS Oficina d'Universitat Saludable i Sostenible

LVUIB Liga de Voleibol de la Universitat de les Illes Balears

SaaS Software as a Service, Software como Servicio

OS Object Storage, Almacenamiento de Objetos

BD Base de Datos

UX User Experience, Experiencia de Usuario

DTO Data Transfer Object, Objeto de Transferencia de Datos

ORM Object-Relational Mapping, Mapeo Relacional de Objetos

ABAC Attribute-Based Access Control, Control de Acceso Basado en Atributos

RESUMEN

[spa] Sports League consiste en un sistema de gestión de ligas deportivas. Permite a cualquier individuo que se registre: crear, organizar y gestionar ligas deportivas de diferentes deportes. La aplicación está pensada para que la gestión de una liga, durante todo su ciclo de vida, ocurra dentro de la aplicación. Algunas de las funcionalidades destacadas que incluye el sistema son: creación de ligas basada en la consecución de fases de distintos tipos, jerarquía de roles dentro de una liga, creación de equipos y solicitudes de unión a estos, asignación de fechas de partidos mediante acuerdos entre los capitanes de los equipos enfrentados, asignación automática de árbitros a partidos mediante algoritmos que equilibran el arbitraje, generación y resolución de incidencias y funcionalidades básicas que se pueden encontrar en cualquier aplicación de gestión de ligas deportivas.

La aplicación está pensada como un sistema multitenencia que sigue una arquitectura cliente-servidor y que permite la comunicación entre los usuarios y el sistema central. Sports League se ha diseñado e implementado como una aplicación full-stack que consta de: una API implementada en Spring Boot-Kotlin, que hace el papel de servidor back-end, conectada a una Base de Datos (BD) PostgreSQL y un Object Storage, Almacenamiento de Objetos (OS) MinIO, ambos locales; y una interfaz multiplataforma implementada con React Native, que permite la interacción de los usuarios con el sistema.

[cat] Sports League consisteix en un sistema de gestió de lligues esportives. Permet a qualsevol individu que es registri: crear, organitzar i gestionar lligues esportives de diferents esports. L'aplicació està pensada perquè la gestió d'una lliga, durant tot el cicle de vida, passi dins l'aplicació. Algunes de les funcionalitats destacades que inclou el sistema són: creació de lligues basada en la consecució de fases de diferents tipus, jerarquia de rols dins d'una lliga, creació d'equips i sol·licituds d'unió a aquests, assignació de dates de partits mitjançant acords entre els capitans dels equips enfrontats, assignació automàtica d'àrbitres a partits mitjançant algorismes que equilibren l'arbitratge, generació i resolució d'incidències i funcionalitats bàsiques que es poden trobar a qualsevol aplicació de gestió de lligues esportives.

L'aplicació està pensada com un sistema multitinença que segueix una arquitectura client-servidor i que permet la comunicació entre els usuaris i el sistema central. Sports League s'ha dissenyat i implementat com una aplicació full-stack que consta de: una API implementada a Spring Boot-Kotlin, que fa el paper de servidor back-end, connectada a una BD PostgreSQL i un OS MinIO, tots dos locals; i una interfície multiplataforma implementada amb React Native, que permet la interacció dels usuaris amb el sistema.



INTRODUCCIÓN

1.1 Estructura del documento

Este documento sirve para entender el proceso de trabajo aplicado para el desarrollo de Sports League. Por ello cada capítulo explica una parte en concreto del proyecto.

- La información referente al contexto del proyecto, motivaciones, objetivos y alternativas se encuentran en el capítulo 1.
- Todo lo referente a las tecnologías disponibles para la implementación del sistema se encuentran en el capítulo 2.
- El proceso de desarrollo al completo queda explicado en el capítulo 3.
- Los resultados obtenidos se explican en el capítulo 4.
- Conclusiones, aprendizajes, aportaciones y trabajo futuro están explicados en el capítulo 5.
- Los anexos del proyecto se encuentran en su capítulo correspondiente.

1.2 Contexto

Como participante activo que he sido en Liga de Voleibol de la Universitat de les Illes Balears (LVUIB) durante los años que se ha celebrado me he dado cuenta de lo costoso que es encontrar un sistema a nivel de aplicación que permita organizar y gestionar una liga deportiva. Suelen ser necesarias muchas funcionalidades avanzadas y personalizadas que llevan a optar por soluciones que acaban necesitando una inversión monetaria, pero que aun así no acaban de convencer a los participantes y usuarios de dicho sistema.

Esta situación es una descripción de lo ocurrido con la LVUIB del año 2025-2026. La experiencia de los alumnos participantes con dicha aplicación resultó en un descontento general debido a: el uso de múltiples aplicaciones para los árbitros, la incorrecta persistencia y tardía sincronización de los resultados de los partidos y la falta de funcionalidades que obligaba a mantener un flujo de gestión híbrido entre la aplicación y la propia Universitat de les Illes Balears (UIB) entre otras razones.

Cabe remarcar que la Oficina d'Universitat Saludable i Sostenible (OUSIS) es el organismo encargado de fomentar y gestionar los eventos deportivos y relacionados con la salud que se celebran dentro del entorno de la UIB, como pueden ser las ligas universitarias de fútbol y pádel. Esta gestión no estuvo tan presente durante el primer año que se celebró la LVUIB, ya que fue un evento fomentado y gestionado por una asociación de alumnos creada al uso para dicho evento. En años posteriores, a medida que la LVUIB fue creciendo en popularidad y participantes, la OUSIS decidió involucrarse de lleno.

Debido a esta situación llegué a la conclusión de que era posible desarrollar un sistema que permitiera una gestión centralizada evitando los problemas comentados y que, además, permitiera utilizar la aplicación como el único punto necesario de gestión.

1.3 Motivación y objetivos

La motivación principal de este Treball Final de Grau (TFG) ha sido la de crear un sistema gestor de ligas deportivas que aporte una propuesta de valor diferenciadora con respecto a las soluciones que existen en el mercado.

Los objetivos principales que han buscado cumplir a través del desarrollo de Sports League es:

1. **Desarrollar el sistema de gestión:** crear un sistema capaz de permitir a usuarios crear y gestionar ligas deportivas, principalmente enfocado a entornos universitarios.
2. **Crear un flujo de trabajo cerrado:** diseñar e implementar el sistema de forma que no sea necesario optar por ninguna herramienta externa al propio sistema para completar un ciclo de vida completo de una liga en la que se participe, ya sea tanto para acordar las fechas de los partidos como para generar incidencias referentes a problemas ocurridos dentro de la liga.
3. **Unificar el sistema para todos los roles:** el sistema debe ser capaz de ofrecer las funcionalidades necesarias para completar la gestión de las ligas para cualquier tipo de usuario participe en estas: jugador, capitán, árbitro y administrador.

1.4 Alternativas actuales

Aunque actualmente podemos encontrar miles de alternativas que aportan las funcionalidades básicas y genéricas necesarias para una correcta gestión de ligas deportivas, prefiero centrarme en las aplicaciones utilizadas en las temporadas anteriores de la LVUIB. Dichas aplicaciones son: Winner y Clupik

Winner se utilizó en el primer y segundo año que se celebró la LVUIB. Es una aplicación que se puede encontrar en cualquier tienda de aplicaciones móviles. Dado que la responsabilidad de gestión cayó principalmente en la asociación de alumnos anteriormente comentada, se optó por una herramienta gratuita que permitía funcionalidades básicas: gestión de equipos, gestión de partidos y visualización de tablas clasificación. En caso de querer más funcionalidades, se debía hacer una pequeña inversión no superior a 10€: una opción asequible y por la que se optó en el segundo año.

En cambio, Clupik se ha estado utilizando durante el tercer año (coincidiendo con el año académico 2025-2026) y ha representado un cambio importante respecto a la primera alternativa, ya que la propia OUSIS fue la responsable de adquirir la herramienta. Esta herramienta se presenta como un Software as a Service, Software como Servicio (SaaS) por el que la OUSIS ha tenido que hacer una inversión en torno a los 600 €. Este coste se debe a que Clupik es una solución profesional, personalizada y enfocada a organismos que necesitan sistemas gestores de ligas deportivas. Podría parecer que Clupik es la herramienta ideal para el caso de la LVUIB, pero todo lo contrario. Esta alternativa ha presentado muchos problemas tanto por parte de los jugadores como de los árbitros y administradores de la liga.

ESTADO DEL ARTE EN EL ÁMBITO TECNOLÓGICO

Para implementar este tipo de sistemas requerimos de tecnologías que nos permitan implementar las funcionalidades que deseamos mediante:

1. Un servidor que hace el papel de API que se conecta a: una base de datos para persistir datos tabulares y relacionales y un almacenamiento de objetos para guardar imágenes de perfil y documentos.
2. Una interfaz multiplataforma que se conecta a la API para mostrar los datos persistidos e interactuar con ellos.

Para implementar la API podemos elegir entre varios lenguajes de programación y sus correspondientes frameworks y librerías: Python con FastAPI o Django, PHP con Laravel, Rust con Tokio y Java/Kotlin con SpringBoot entre otros.

Para la BD encontramos opciones como MySQL, PostgreSQL, SQL Server, Oracle, MariaDB si queremos un sistema relacional. Aunque también podemos optar por MongoDB, Redis o Cassandra, entre otros, si preferimos sistemas no relacionales.

Para el OS la opción más utilizada es el almacenamiento en la nube que ofrece Amazon S3. Una alternativa a Amazon S3 podría ser Cloudflare R2. Estas soluciones alojadas en la nube acarrearán un costo si el volumen de datos y consultas es alto. Lo bueno es que podemos optar por versiones autoalojadas para tener más privacidad y control sobre los recursos, además de eliminar cualquier costo. La última opción, pero menos recomendada, sería guardar las imágenes y documentos en una carpeta del sistema de ficheros del servidor donde se aloja el back-end.

Finalmente, para implementar la interfaz de usuario gráfica multiplataforma podemos optar por las siguientes tecnologías: React Native, Ionic y Quasar, basadas en tecnologías web; o Flutter y Kotlin Multiplataforma, basadas en tecnologías nativas.

2.1 Tecnologías utilizadas

Entre todas estas tecnologías se han seleccionado las más convenientes y útiles dados ciertos criterios, algunos de ellos objetivos, pero otros completamente subjetivos.

La interfaz gráfica está implementada utilizando React Native. La razón principal de esta elección se debe a que ya he tenido experiencia desarrollando páginas web con React. Esto hace que no sea necesario aprender nuevos lenguajes, sino que simplemente puedo aplicar el conocimiento que ya tengo junto a una tecnología que aporta el desarrollo multiplataforma de forma nativa. Así mismo, he utilizado el empaquetador de Expo, altamente recomendado en la propia documentación de React Native. De esta forma, el desarrollo es mucho más cómodo, y es posible compilar el código a cualquier plataforma sin problemas.

El servidor back-end se ha implementado en Spring Boot-Kotlin. La elección de Kotlin se debe a: la baja curva de aprendizaje por haber tenido experiencia con otro lenguaje basado en la JVM, como es Java; y en la practicidad y facilidad de programación y entendimiento que ofrece Kotlin como herramienta de codificación. Se ha elegido Spring Boot porque aporta sencillez a la hora de crear aplicaciones que escuchan y sirven peticiones. Es capaz de soportar una alta carga de usuarios concurrentes manteniendo simplicidad en el código.

La BD ha sido implementada utilizando el motor de PostgreSQL. Podría entenderse que la mejor opción era MySQL o MariaDB, ya que son las tecnologías que se enseñan dentro del Grau d'Enginyeria Informàtica (GEIN). Dichas opciones se tuvieron en cuenta, pero a decisión personal, finalmente se optó por PostgreSQL debido a un interés en aprender a utilizar esta tecnología. Según Stack Overflow es el motor más popular dentro del mundo del desarrollo abarcando más de la mitad de la cuota de mercado Stack Overflow Survey. Esto la convierte en una tecnología que vale la pena investigar y aprender a usar.

Finalmente, para el OS se ha decidido utilizar un contenedor de Docker de MinIO que se ejecuta localmente en el servidor junto al back-end. Las razones principales han sido la privacidad, el control y el nulo coste de uso de la herramienta para el caso que nos acontece. MinIO permite tener un OS con control total de los recursos sin necesidad de optar por el almacenamiento en la nube. Además, la API utilizada en Spring Boot es completamente compatible con la de Amazon S3, por lo que si en algún momento fuera necesario escalar el almacenamiento y llevarlo a la nube, tan solo haría faltar cambiar algunas variables de entorno.

PROCESO DE DESARROLLO

En este capítulo veremos todos los pasos que han sido necesarios abarcar para completar el TFG, desde la recogida de requisitos hasta la verificación del correcto funcionamiento del sistema.

Para completar el proyecto se ha optado por un desarrollo en cascada. Este enfoque nos permite asegurar que un paso está completo antes de pasar al siguiente. No se ha optado por metodologías ágiles, ya que incurren en procesos no tan necesarios cuando no tenemos un equipo de trabajo y la organización del proyecto recae en una sola persona.

El primer paso ha sido la recogida y análisis de requisitos. Seguidamente, se ha hecho el diseño tanto gráfico como arquitectónico de todos los servicios. Se ha continuado con la implementación en el siguiente orden: implementación completa del back-end junto con la BD y el OS seguida de la implementación del front-end. Por último, se han creado los test sobre el back-end para verificar el correcto funcionamiento del sistema.

3.1 Análisis y diseño

3.1.1 Recogida de requisitos

Los requisitos funcionales han sido generados basándonos en las opiniones que tenían los participantes de la LVUIB acerca del sistema que se ha estado utilizando: Clupik. Para completar la recogida de dichos requisitos se optó por la distribución de formularios de opinión y sugerencias. Las preguntas formuladas (que se pueden encontrar en los anexos) se agrupan en 4 formularios distintos según el rol que tuviera el participante dentro de la liga: jugador, capitán, árbitro y administrador.

Todos los formularios tenían en común, cuestiones relacionadas con: la frecuencia de uso de la aplicación, la plataforma en la que mayormente se utiliza la aplicación y una valoración subjetiva sobre diferentes aspectos y funcionalidades que no funcionan o que encuentran a faltar. Por otra parte, cada formulario contenía una pequeña sec-

ción de cuestiones concretas relacionadas con el rol al cual se enfocaba el formulario, además de un listado de funciones deseables.

El formulario de jugadores no contenía ninguna cuestión específica, tan solo las anteriormente comentadas.

El formulario de capitanes se centraba en el manejo de la aplicación a nivel de capitán: gestión de su propio equipo y sus miembros, contacto con otros capitanes para el acuerdo de las fechas de los partidos

El formulario de árbitros se centraba en el manejo a nivel de árbitro: arbitraje de partidos, sanciones a los jugadores durante un partido, registro de los resultados y firma de actas.

Y el formulario de administradores se centraba en el manejo a nivel de administrador: gestión de la información de equipos, jugadores, partidos, árbitros, incidencias y sanciones, todo ello disponible dentro de la propia aplicación.

Los formularios fueron distribuidos a través de WhatsApp a los distintos grupos objetivos y se esperó a que hubiera una cantidad significativa de respuestas.

3.2 Análisis

Las respuestas presentadas en los formularios concluyeron en los siguientes resultados:

A partir de estos datos se generaron los siguientes requisitos funcionales, separados según la entidad a la que están asociados. Hemos utilizado esta separación, ya que de este modo, tendremos más claro como diseñar la BD.

3.2.1 Reunión con la OUSIS

Al mismo tiempo que se empezó con la recogida de requisitos, fui contactado por la OUSIS. Habían escuchado que estaba desarrollando este proyecto y decidieron contactar conmigo para aclarar ciertas dudas que tenían.

Tuvimos dos reuniones en la que no se hicieron ningún acta de reunión. En la primera, estuve acompañado por Raimon Massanet y consistió en una ronda de preguntas por parte de los responsables de la OUSIS: Joan Mercant Requena y Olalla Garcia Taibo. Fue necesario aclarar que las encuestas eran de carácter completamente anónimo y no se recogía ningún dato personal.

En la segunda reunión, acompañado por Antonio Crespi, se estableció un contacto profesional para utilizar los resultados del proyecto como una base para implementar un nuevo sistema de gestión de actividades deportivas dentro de la UIB. Algunas de sus peticiones fueron: poder utilizar la aplicación para gestionar otras actividades que no sean ligas deportivas y extender el uso de la aplicación a más deportes, ya que en un principio la aplicación estaba enfocada únicamente a voleibol. Debido a esto, se intentó que todo el proyecto fuera capaz de soportar estos requisitos a futuro, pero sin alterar el alcance base del sistema.

3.2.2 Requisitos funcionales

A partir de los resultados de las encuestas y las reuniones con la OUSIS se sintetizaron los siguientes requisitos funcionales y no funcionales:

Cuadro 3.1: Requisitos funcionales del sistema

Código	Entidad	Descripción
RF-USR-1	Usuario	El sistema debe permitir a los usuarios poder crear una cuenta para registrar sus datos en el sistema.
RF-USR-2	Usuario	El sistema debe permitir a los usuarios registrar la siguiente información: nombre completo, email, contraseña, sexo, referencias de las distintas licencias arbitrales que pueda tener un usuario y foto de perfil.
RF-USR-3	Usuario	El sistema debe permitir que los usuarios puedan indicar el deporte al que corresponden sus distintas licencias arbitrales.
RF-USR-4	Usuario	El sistema debe permitir a los usuarios iniciar sesión con una cuenta ya creada dadas sus credenciales: nombre de usuario o email y contraseña.
RF-USR-5	Usuario	El sistema debe permitir a los usuarios con sesión iniciada crear ligas deportivas. El usuario que crea una liga se convierte automáticamente en administrador de la liga.
RF-USR-6	Usuario	El sistema debe permitir a los usuarios sin cuenta poder ver ligas ya creadas.
RF-USR-7	Usuario	El sistema debe permitir que los usuarios puedan almacenar su firma como una imagen que pueda ser reutilizada para firmar en los partidos en caso de ser capitán o árbitro.
RF-LIG-1	Liga	El sistema debe permitir crear ligas con la siguiente información: nombre, descripción, icono, banner, ubicación, fecha de inicio, fecha de fin y fecha máxima para inscribirse.
RF-LIG-2	Liga	El sistema debe permitir configurar los siguientes parámetros dentro de una liga: categoría (Masculina, Femenina o Mixta), número de integrantes femeninos mínimos en caso de que la categoría sea mixta, número mínimo y máximo de integrantes en cada equipo y duración de las jornadas en semanas.

Continúa en la página siguiente

Cuadro 3.1 – continuación de la página anterior

Código	Entidad	Descripción
RF-LIG-3	Liga	El sistema debe permitir implementar un sistema de roles dentro de las ligas: Administrador, Árbitro, Capitán y Jugador.
RF-LIG-4	Liga	El sistema debe permitir a los usuarios con sesión iniciada unirse a ligas creadas. Al unirse se convierte en Jugador dentro de esa liga.
RF-LIG-5	Liga	El sistema debe permitir que los participantes de una liga puedan crear incidencias dentro de esta. Las incidencias deben ir acompañadas de una descripción.
RF-LIG-6	Liga	El sistema debe permitir que los administradores de una liga puedan expulsar a los participantes de esta.
RF-LIG-7	Liga	El sistema debe permitir que una liga esté conformada por distintas fases.
RF-LIG-8	Liga	El sistema debe permitir que un usuario pueda elegir ver únicamente las ligas en las que está participando.
RF-EQU-1	Equipo	El sistema debe permitir a los usuarios dentro de una liga crear y enviar solicitudes a los administradores para poder crear equipos dentro de la liga.
RF-EQU-2	Equipo	El sistema debe poder permitir crear equipos con la siguiente información: nombre, iniciales, descripción, lema, color primario, color secundario y escudo.
RF-EQU-3	Equipo	El sistema debe permitir al administrador de una liga aceptar o rechazar solicitudes de creación de equipo. En caso de ser rechazada se debe poder incluir un mensaje con la justificación pertinente.
RF-EQU-4	Equipo	El sistema debe permitir que, al aceptar una solicitud de creación de equipo, el solicitante se convierta automáticamente en el capitán del equipo solicitado.

Continúa en la página siguiente

Cuadro 3.1 – continuación de la página anterior

Código	Entidad	Descripción
RF-EQU-5	Equipo	El sistema debe permitir que los jugadores que no tengan equipo puedan enviar una solicitud al capitán de un equipo para poder unirse a este.
RF-EQU-6	Equipo	El sistema debe permitir que los capitanes puedan aceptar o rechazar las solicitudes de unión de su equipo.
RF-EQU-7	Equipo	El sistema debe permitir que los capitanes de los equipos puedan enviar solicitudes a jugadores sin equipo para que se unan a este.
RF-EQU-8	Equipo	El sistema debe permitir que los jugadores puedan aceptar o rechazar las solicitudes para unirse a un equipo.
RF-EQU-9	Equipo	El sistema debe permitir que los capitanes de un equipo puedan expulsar a los jugadores de este.
RF-EQU-10	Equipo	El sistema debe permitir que los jugadores puedan elegir un número de dorsal dentro de su equipo que no se haya elegido antes.
RF-EQU-11	Equipo	El sistema debe permitir que el capitán de un equipo pueda eliminar el equipo que lidera.
RF-ROL-1	Rol	El sistema debe permitir que los participantes de una liga puedan tener uno o más roles asignados.
RF-ROL-2	Rol	El sistema debe permitir que el creador de una liga pueda asignar el rol de Administrador a otros participantes de esta. De este modo se puede repartir el trabajo que conlleva gestionar la liga.
RF-ROL-3	Rol	El sistema debe permitir que los jugadores de una liga puedan crear solicitudes para que los administradores les puedan asignar el rol de Árbitro en dicha liga.

Continúa en la página siguiente

Cuadro 3.1 – continuación de la página anterior

Código	Entidad	Descripción
RF-ROL-4	Rol	El sistema debe permitir que los administradores rechacen o acepten las solicitudes de los jugadores para convertirse en árbitros. Dicha respuesta y solicitud cuenta con la misma información que las solicitudes para crear equipos.
RF-ROL-5	Rol	El sistema debe permitir que los administradores puedan retirar el rol de Árbitro a los participantes árbitros que vea conveniente.
RF-ROL-6	Rol	El sistema debe permitir que los administradores puedan eliminar un equipo.
RF-FAS-1	Fase	El sistema debe permitir configurar cada fase con la siguiente información: nombre, fecha de inicio, fecha de fin, formato y su orden dentro de la secuencia de fases.
RF-FAS-2	Fase	El sistema debe permitir que cada fase pueda ser uno de los siguientes formatos: Torneo y Clasificatorio. En el formato Clasificatorio se debe especificar el número de ganadores que tendrá. Podrían añadirse nuevos formatos en un futuro.
RF-FAS-3	Fase	El sistema debe permitir que cada fase esté compuesta por una o más Rondas.
RF-RON-1	Ronda	El sistema debe permitir que para cada Ronda se pueda establecer el día en que empieza dicha Ronda.
RF-RON-2	Ronda	El sistema debe permitir que en cada ronda se celebren diversos partidos.
RF-RON-3	Ronda	El sistema debe permitir que los usuarios con rol de Administrador de una liga establezcan la disponibilidad de la ubicación de juego para cada Ronda.
RF-DIS-1	Disponibilidad	El sistema debe permitir que la disponibilidad de una ronda se establezca como un conjunto de franjas horarias.

Continúa en la página siguiente

Cuadro 3.1 – continuación de la página anterior

Código	Entidad	Descripción
RF-DIS-2	Disponibilidad	El sistema debe permitir que en cada franja horaria se pueda especificar el día, la hora y la duración de la franja.
RF-DIS-3	Disponibilidad	El sistema debe permitir que los administradores de una liga puedan establecer la disponibilidad de la ubicación de juego para cada ronda.
RF-DIS-4	Disponibilidad	El sistema debe permitir que cada usuario, en especial los capitanes y los árbitros, puedan establecer su disponibilidad para cada ronda respecto a la disponibilidad establecida por los administradores para la ubicación de juego.
RF-PAR-1	Partido	El sistema debe permitir que los partidos se generen de forma automática dependiendo del formato de la Fase.
RF-PAR-2	Partido	El sistema debe permitir que en cada partido se enfrenten un equipo local y un equipo visitante.
RF-PAR-3	Partido	El sistema debe permitir que las asignaciones de los equipos que se enfrentan en los partidos se pueda hacer de forma automática o manual.
RF-PAR-4	Partido	El sistema debe permitir que la fecha de un partido se establezca mediante un sistema de negociación entre los capitanes de los equipos que deben enfrentarse.
RF-PAR-5	Partido	El sistema debe permitir que los administradores de una liga puedan fijar la fecha de un partido que no se haya fijado.
RF-PAR-6	Partido	El sistema debe permitir que a cada partido se pueda asignar un árbitro. El árbitro se puede asignar de forma manual por parte de los Administradores o de manera automática.

Continúa en la página siguiente

Cuadro 3.1 – continuación de la página anterior

Código	Entidad	Descripción
RF-PAR-7	Partido	El sistema debe permitir que la forma automática para asignar un árbitro a un partido se haga mediante un algoritmo que, para un árbitro con una cierta disponibilidad, se le asigne un partido cuya fecha coincida con alguna de las franjas disponibles del árbitro. El algoritmo también debe tener en cuenta la cantidad de partidos arbitrados por cada árbitro para poder repartir el arbitraje de manera equitativa.
RF-PRO-1	Propuesta	El sistema debe permitir que el proceso de negociación de la fecha de un partido consista en lo siguiente: el capitán del equipo local crea y envía una primera propuesta con la franja horaria disponible deseada al capitán del equipo visitante. El capitán del equipo visitante puede decidir aceptar o rechazar la propuesta, si la rechaza debe enviar una contra-propuesta. Este proceso se repite hasta que alguno de los dos acepte la propuesta.
RF-RES-1	Resultado	El sistema debe permitir que se puedan guardar un Resultado para cada partido.
RF-RES-2	Resultado	El sistema debe permitir que cada resultado contenga la siguiente información: puntuación del equipo local, puntuación del equipo visitante, observaciones encontradas durante el partido y firmas de los árbitros y capitanes antes y después del partido.
RF-RES-3	Resultado	El sistema debe permitir que cada capitán y árbitro pueda firmar el acta de un partido desde su propio dispositivo.
RF-RES-4	Resultado	El sistema debe permitir que un partido se divida en distintos Periodos.
RF-RES-5	Resultado	El sistema debe permitir que cada periodo guarde la siguiente información: orden numérico del periodo dentro del partido, tipo de periodo según el deporte (set, mitad, cuarto, etc.) y puntuación de los dos equipos para ese periodo.

Continúa en la página siguiente

Cuadro 3.1 – continuación de la página anterior

Código	Entidad	Descripción
RF-PER-1	Periodo de Partido	El sistema debe permitir que cada Periodo contenga: la puntuación del equipo local y del equipo visitante para ese periodo, orden numérico del periodo dentro del partido y tipo de periodo según el deporte.
RF-PER-2	Periodo de Partido	El sistema debe permitir que cada periodo pueda tener Eventos.
RF-EVE-1	Evento de Partido	El sistema debe permitir que para cada evento se guarde la siguiente información: el tiempo en el que se genera respecto al inicio del partido, puntuaciones del equipo local y visitante para ese periodo y qué equipo ha sido artífice de dicho evento.
RF-EVE-2	Evento de Partido	El sistema debe permitir que cada evento pueda ser uno de tres tipos distintos: sustitución, sanción o tiempo muerto. Para la sustitución se debe especificar los dos jugadores que se intercambian. En la sanción se debe especificar una descripción y el tipo de sanción. Para los tiempos muertos se debe especificar la duración de dicho tiempo.

Cuadro 3.2: Requisitos no funcionales del sistema

Código	Tipo	Descripción
RNF-USA-1	Usabilidad	La aplicación móvil debe permitir a un árbitro registrar eventos y cerrar un acta en un máximo de 3 a 5 clics desde la pantalla principal.
RNF-USA-2	Usabilidad	La interfaz debe contar con un contraste adecuado para garantizar la legibilidad en entornos exteriores (pistas al aire libre con mucha luz).
RNF-SEG-1	Seguridad	El acceso a los endpoints críticos (como forzar un horario o modificar roles) debe estar protegido mediante autenticación por token (JWT) y validación estricta de roles y atributos: Attribute-Based Access Control, Control de Acceso Basado en Atributos (ABAC).

Continúa en la página siguiente

Cuadro 3.2 – continuación de la página anterior

Código	Tipo	Descripción
RNF-SEG-2	Seguridad	Las contraseñas de los usuarios no se almacenarán en texto plano bajo ninguna circunstancia; se utilizará un algoritmo de "hashing" fuerte (como "bcrypt").
RNF-SEG-3	Seguridad	Sanitización de entradas: Todos los datos enviados desde la aplicación móvil hacia la API deben ser validados y sanitizados para prevenir ataques de inyección SQL (gestionado a través del Object-Relational Mapping, Mapeo Relacional de Objetos (ORM) de Spring JPA) y ataques XSS (Cross-Site Scripting).
RNF-DEP-1	Fiabilidad	El sistema debe garantizar la atomicidad de las operaciones críticas (como el guardado del acta final). Si la conexión se pierde a mitad del envío, la base de datos debe revertir los cambios para evitar estados inconsistentes (actas a medias).
RNF-EFI-1	Eficiencia (Espacio)	Las imágenes subidas al sistema (como firmas de capitanes, fotos de perfil o logos de equipos) deben comprimirse para no superar un peso máximo de 200KB por imagen.
RNF-EFI-2	Eficiencia (Rendimiento)	Las consultas de alta frecuencia, como la visualización de la clasificación, deben resolverse en menos de 200 ms desde el back-end, apoyándose en la base de datos optimizada (vistas materializadas).
RNF-EFI-3	Eficiencia (Rendimiento)	La aplicación móvil debe arrancar y mostrar la pantalla de inicio (Dashboard) en menos de 3 segundos en condiciones de red estándar (4G/Wifi).
RNF-OPE-1	Operacional	Las constantes del sistema de competición (como puntos por victoria, duración de jornadas) deben ser configurables desde la base de datos sin necesidad de recompilar el código del servidor.

Continúa en la página siguiente

Cuadro 3.2 – continuación de la página anterior

Código	Tipo	Descripción
RNF-OPE-2	Pila Tecnológica	Para garantizar la homogeneidad y mantenibilidad del sistema, el back-end debe desarrollarse estrictamente en Kotlin con Spring Boot (exponiendo API REST) y el front-end móvil en React Native con Expo. La persistencia de datos relacionales recaerá exclusivamente en PostgreSQL, mientras que la persistencia de datos no relacionales recaerá en un servicio Object Storage - S3 implementado con MinIO.
RNF-OPE-3	Contenerización	El entorno del servidor debe estar empaquetado mediante contenedores de Docker para asegurar que la aplicación funcione de manera idéntica tanto en la máquina local de desarrollo como en el servidor de producción.
RNF-DES-1	Desarrollo	Control de Versiones: Todo el código fuente del proyecto (front-end, back-end y scripts de base de datos) debe estar centralizado y versionado utilizando Git, alojado en un repositorio en la nube como GitHub.
RNF-DES-2	Desarrollo	Estrategia de Ramas (Branching): El equipo de desarrollo debe seguir un flujo de trabajo estructurado. Existirá una rama "main" que contendrá todo el código desarrollado.
RNF-DES-3	Desarrollo	Pruebas Automatizadas (Pruebas Unitarias): La lógica de negocio crítica en Kotlin (por ejemplo, el cálculo de los puntos de la liga o la máquina de estados de los partidos) debe estar cubierta por pruebas unitarias independientes de la base de datos.
RNF-DES-4	Desarrollo	Pruebas Automatizadas (Pruebas de Integración): Se deben incluir pruebas que verifiquen el correcto funcionamiento de los endpoints principales y sus consultas contra una base de datos de prueba (por ejemplo, usando "Testcontainers." o una base de datos en memoria).

Continúa en la página siguiente

Cuadro 3.2 – continuación de la página anterior

Código	Tipo	Descripción
RNF-DES-5	Desarrollo	Gestión de variables de entorno: Ninguna credencial, token secreto o contraseña de base de datos debe subirse al repositorio de código. Todas las configuraciones sensibles deben inyectarse mediante variables de entorno (archivos .env en React Native y application.yml parametrizado en Spring Boot).
RNF-DES-6	Desarrollo	El diseño de la API REST debe estar documentado mediante el estándar Open API (Swagger), actuando como único contrato y fuente de la verdad para sincronizar el trabajo entre el front-end y el back-end.
RNF-DES-7	Desarrollo	El código fuente del back-end debe seguir los principios de separación por capas (Controlador, Servicio, Repositorio) para asegurar que la lógica de negocio esté aislada y sea fácilmente verificable. Además, debe aplicarse una "separación vertical" por entidades para mantener dichas funcionalidades claramente separadas.

3.3 Diseño

En esta sección nos centraremos en el diseño de: la BD, la interfaz de usuario y el contrato API.

3.3.1 Diseño de la BD

El diseño de la BD ha sido un derivado directo de los requisitos funcionales y muestra cómo las entidades se relacionan entre ellas a partir de los requisitos. Se ha utilizado la herramienta de draw.io para crear un diagrama de clases UML que muestra la estructura de la BD. En el enlace Diagrama UML - draw.io podemos ver el diagrama de mucho más detallado.

En el diagrama se pueden apreciar: las clases definidas con sus atributos correspondientes y las relaciones que relacionan dichas clases. Cabe aclarar que se pueden ver algunos nombre de clases repetidos seguido de un asterisco "*". Dicho formato indica que la clase ya ha sido definida y que simplemente se usa como una referencia". Se ha decidido aplicar este formato por claridad visual. De esta forma evitamos que las relaciones que conectan dos clases muy separadas no estorben en la lectura del diagrama.

3.3.2 Diseño de la interfaz

Es necesario pensar en cómo los usuarios pueden interactuar con el sistema para cumplir con requisitos, es decir, necesitamos definir la User Experience, Experiencia de Usuario (UX).

Para el diseño de la interfaz se ha optado por usar la herramienta de Figma al ser un estándar en el mundo del diseño sin coste. La versión preliminar de la interfaz se puede ver a través del siguiente enlace: [Mockup en Figma](#)

3.3.3 Diseño del contrato API

El desarrollo del sistema se ha enfocado como un proceso API-first. Lo que quiere decir, que el primer paso ha sido definir un contrato API que funciona como única fuente de la verdad y del que se derivan todos los Data Transfer Object, Objeto de Transferencia de Datos (DTO) utilizados tanto en el back-end como en el front-end, para estandarizar la estructura de los mensajes que permiten la comunicación entre estos dos servicios. Para redactar el contrato se ha hecho uso del estándar Open API ampliamente usado en la industria del desarrollo.

Además, el contrato se ha separado por entidades para poder separar todo y mejorar el entendimiento para los desarrolladores. El versionado del contrato se ha llevado a cabo mediante Git y se ha subido a su propio repositorio. De esta forma, tanto el back-end como el front-end pueden utilizar el repositorio del contrato como un submódulo y operar siempre con la última versión.

El contenido en detalle del contrato API se puede encontrar como anexo en el apartado correspondiente.

3.4 Arquitectura del sistema

Las siguientes imágenes que representan los tres primeros niveles de un modelo C4 explican en detalle la arquitectura establecida para Sports League:

El nivel 1 muestra la relación de la aplicación con los usuarios y servicios externos.

El nivel 2 indica cómo interactúan los diferentes sistemas entre ellos.

El nivel 3 indica cómo se ha estructurado el back-end: el núcleo del sistema.

3.5 Planificación

La planificación del proyecto puede apreciarse mediante el siguiente cronograma:

3.6 Implementación

La BD se ha implementado mediante los siguientes archivos .sql que dividen la creación de la BD en cuatro pasos:

1. **V1__init_schema:** crea todas las tablas necesarias para representar de forma correcta el diagrama UML.

3. PROCESO DE DESARROLLO

2. **V2__add_soft_delete_triggers:** crea los disparadores necesarios para poder aplicar un sistema de borrado lógico sobre los recursos cuya eliminación física acarrearía estados no deseados dentro del sistema.
3. **V3__create_leaderboard_view:** crea la vista encargada de calcular las puntuaciones de los equipos para cada liga, grupo y jornada.
4. **V99__insert_initial_data:** inserta los datos necesarios para que el sistema pueda funcionar de forma correcta: los deportes disponibles, roles de usuario y una configuración y sistema de puntuación por defecto para ligas de voleibol.

Para la generación de los DTO en todos los servicios se han utilizado herramientas específicas para la generación de clases a partir de un contrato API. En el back-end, se ha utilizado la librería ...; y en el front-end, el paquete

CAPÍTULO



4

CONCLUSIONES

- 4.1 Aprendizaje**
- 4.2 Aporte**
- 4.3 Trabajo futuro**



APÉNDICE

A.1 Formularios de opinión

A.2 Diseño base de datos

A.3 Contrato API

BIBLIOGRAFÍA